

The FreeBayes realignment and allele-construction pipeline

Annotated source reference — FreeBayes 1.3.10

Source references are given as file:line against the FreeBayes 1.3.10 source tree.

Contents

1 Purpose and audience	1
2 Where this sits in the pipeline	2
3 Part I — Local realignment: <code>LeftAlign.{h,cpp}</code>	2
3.1 The problem it solves	2
3.2 Invocation and the “stable” wrapper	3
3.3 One pass of <code>leftAlign</code> (<code>LeftAlign.cpp:25</code>)	3
3.4 <code>countMismatches</code> (<code>LeftAlign.cpp:345</code>)	5
3.5 Worked example	5
4 Part II — Allele construction feeding realignment	5
4.1 <code>updateAlignmentQueue</code> (<code>AlleleParser.cpp:1954</code>): intake and the filter gauntlet	5
4.2 <code>registerAlignment</code> (<code>AlleleParser.cpp:1329</code>): CIGAR → alleles	6
4.3 <code>makeAllele</code> (<code>AlleleParser.cpp:1171</code>): one allele + two side effects	7
4.4 <code>addAllele</code> / <code>clumpAlleles</code> / <code>mergeAllele</code> : complex-event assembly	7
4.5 <code>getAlleles</code> (<code>AlleleParser.cpp:3685</code>): bucketing	8
5 Part III — <code>buildHaplotypeAlleles</code> (<code>AlleleParser.cpp:3214</code>)	8
5.1 Why it exists	8
5.2 Seeding the window length (:3223–3239)	8
5.3 The growth loop (:3245–3310)	9
5.4 Worked example: several nearby indels	9
5.5 <code>fitHaplotype</code> (<code>AlleleParser.cpp:3040</code>): squashing a read into one allele	9
5.6 Complete vs partial observations	10
5.7 Re-derivation, what <code>sort/unique</code> actually deduplicates, and the <code>LEN</code> bug precisely	10
6 Coordinate systems	12
7 Known fragilities and non-determinism	12
8 Appendix — parameter defaults touching this pipeline	13
9 Implications for <code>panFreebayes</code>	14

1 Purpose and audience

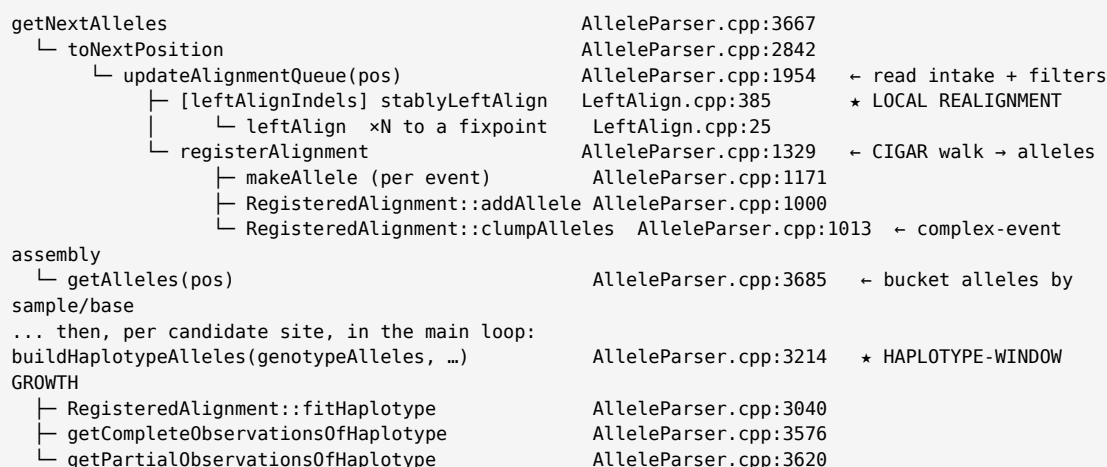
This document explains, in enough detail to stand alone, **how FreeBayes turns a BAM alignment into candidate alleles** — with the local indel realignment step (`LeftAlign.{h,cpp}`) as the centre-piece, and the allele-construction code that feeds it and consumes its output (`registerAlignment`, `makeAllele`, `RegisteredAlignment::clumpAlleles`, `AlleleParser::buildHaplotypeAlleles`).

The emphasis is on **why**: the algorithmic reasoning, the invariants each step maintains, and the places where the implementation is fragile. A reader who has never seen the FreeBayes source should be able to follow the realignment algorithm end to end from this text.

It deliberately stops before the Bayesian genotyping math (data likelihoods, the genotype-combination search, the posterior) — that is a separate subsystem (`DataLikelihood.cpp`, `Genotype.cpp`, `Marginals.cpp`) and a separate document.

2 Where this sits in the pipeline

FreeBayes’s main loop (`freebayes.cpp:117`) processes one reference position at a time: for every position the driver pulls in the reads that now overlap it, converts each read to a list of alleles, widens the analysis window over any indels/repeats, and only then hands a clean set of candidate alleles to the genotyper.



Listing 1: The realignment/allele-construction call graph. ★ marks the two subsystems this document covers.

Two independent “realignment” ideas are in play, and it helps to keep them apart:

Local realignment (`LeftAlign.cpp`) a purely **mechanical**, per-read transform of the CIGAR string. It moves indels to their leftmost equivalent position and merges adjacent ones. It never changes which bases align to which — only the **representation** of gaps. It does not look at other reads.

Haplotype-window construction (`buildHaplotypeAlleles`) a **multi-read** step that decides how wide a window to genotype (so that an indel inside a tandem repeat is compared against the reference over the whole repeat, not just the indel), and re-expresses every overlapping read as a single allele spanning that window.

Local realignment exists so that the second step, and the genotyper after it, see the **same** indel from different reads written the **same** way.

3 Part I — Local realignment: `LeftAlign.{h,cpp}`

3.1 The problem it solves

An aligner is free to place a gap anywhere within a repetitive stretch without changing the alignment score. Given reference `...GAAAAT...` and a read missing one A, all of these CIGARs are score-equivalent:

```

ref   G A A A A T
read  G A A A - T  → 1M... could be written as the gap after any of the 4 A's
read  G A - A A T

```

Different reads covering the same true deletion can therefore arrive with the deletion recorded at different positions. To FreeBayes — which groups observations by (**position**, **type**, **sequence**) — those look like **different** alleles, each with too little support to call. The fix is a canonical form:

always shift an indel as far left as it can go without introducing a mismatch. Every read then agrees, the observations pile up at one position, and the allele becomes callable.

This is the same normalisation `bcftools norm / vt normalize` apply to VCF records; FreeBayes does it up front, on reads.

3.2 Invocation and the “stable” wrapper

`updateAlignmentQueue` calls it once per read, immediately after the read passes the intake filters and before any allele is extracted (`AlleleParser.cpp:2055`):

```
if (parameters.leftAlignIndels) { // default: ON (-O disables)
    int length = currentAlignment_end_position - currentAlignment.POSITION + 1;
    stablyLeftAlign(currentAlignment,
                    currentSequence.substr(currentSequencePosition(currentAlignment),
length));
}
```

The reference slice passed in spans exactly the read’s aligned footprint (`POSITION` to `ENDPOSITION`), taken from the in-memory `currentSequence` (the whole contig; see §6). `stablyLeftAlign` (`LeftAlign.cpp:385`) then calls `leftAlign` repeatedly:

```
if (!leftAlign(alignment, refSeq, debug)) return true; // already canonical
while (leftAlign(alignment, refSeq, debug) && --maxiterations > 0) { }
return maxiterations > 0; // false ⇒ never stabilised (caller warns, keeps read)
```

One pass of `leftAlign` is not guaranteed to reach the fixed point: shifting indel *A* left can open room for indel *B* to shift, and merging two indels can create a new one that is itself shiftable. The pipeline call passes no explicit limit, so `maxiterations` takes the header default of 20 (`LeftAlign.h:118`); the stand-alone `bamleftalign` tool passes 50. In practice two or three passes suffice. A read that does not stabilise within the limit keeps its last intermediate CIGAR — the pipeline call (`AlleleParser.cpp:2057`) discards `stablyLeftAlign`’s return value, so there is no warning; only `bamleftalign` checks it.

Under `VERBOSE_DEBUG`, `stablyLeftAlign` also asserts an **invariant**: `countMismatches` before == after (`LeftAlign.cpp:403`). Left-alignment must never change the number of mismatched bases — if it does, the realignment was illegal and the process aborts. This is the single most important correctness check in the file, and it is compiled out of release builds.

3.3 One pass of `leftAlign` (`LeftAlign.cpp:25`)

3.3.1 Step 0 — parse the CIGAR into an indel list

The pass walks the CIGAR once (:44–81), maintaining `sp` (position in the reference slice) and `rp` (position in the read). It emits an `FBIndelAllele` (`IndelAllele.h`) for every D, N (splice, treated as an un-shiftable deletion) and I operation:

```
class FBIndelAllele {
    bool    insertion;    // I vs D/N
    int     length;
    int     position;     // 0-based, in the reference slice
    int     readPosition; // 0-based, in the read
    string  sequence;     // the inserted read bases, or the deleted reference bases
    bool    splice;       // N: never shifted, never merged
};
```

It also builds two “gapped” strings — `alignedReferenceSequence` and `alignmentAlignedBases` — with - filled in at gap positions and * for soft clips. These are only used for debug printing; the algorithm works on the `FBIndelAllele` list plus the raw `referenceSequence` and `alignmentSequence`.

Soft clips (S) are recorded as `softBegin/softEnd` and their lengths are kept so the final CIGAR can be reconstructed with them intact. Hard clips (H) are ignored — those bases are not in the read.

If the list is empty (`indels.empty()`), the pass returns `false` immediately: nothing to realign.

3.3.2 Step 1 — shift each indel left “by repeats” (:98–143)

For each indel, left to right, try to move it left by a divisor of its own length. The outer loop runs `i` over 1, 2, ... restricted to divisors of `indel.length` (:140–142), and for each `i` the inner `while` (:126–139) slides the indel left in steps of `i` as long as *all* of these hold:

- `steppos >= 0 && readsteppos >= 0` — we have not run off the left end;
- `!indel.splice` — splices never move;
- `indel.sequence == referenceSequence.substr(steppos, indel.length)` — the `indel.length` reference bases immediately to the left are identical to the indel sequence;
- `indel.sequence == alignmentSequence.substr(readsteppos, indel.length)` — and the read bases there are too;
- we would not collide with the previous (already-shifted) indel (:131–133): for an insertion, `steppos >= previous->position`; for a deletion, `steppos >= previous->position + previous->length`.

The middle two conditions are the “no new mismatch” guarantee expressed concretely: moving a `k`-base indel left by `k` is free **iff** the `k` bases it jumps over are a copy of the indel. Restricting `i` to divisors of the length is the standard trick for handling repeat units that do not divide evenly (e.g. a 6-base deletion of `ATATAT` shifts cleanly by 2).

3.3.3 Step 2 — shift left “by exchangeable flanking bases” (:159–176)

Step 1 only moves an indel by whole repeat units. Step 2 handles the single-base homopolymer / tandem case where the **last** base of the indel sequence equals the reference base just to its left:

GTTACGTT		GTTACGTT	
GT-----T	→	G-----TT	(the deletion's trailing T is "rotated" to the front)

The `while` at :161 slides the indel one base left at a time as long as `alignmentSequence[readsteppos] == referenceSequence[steppos]` *and* that base equals the indel’s last character, rotating `indel.sequence` (:171, `s = s.back() + s[0..n-1]`) as it goes, again guarding against the previous indel. This is what actually left-normalises homopolymer indels.

3.3.4 Step 3 — pull floating indels together (:185–238)

Before rebuilding the CIGAR, adjacent same-class indels that **could** be merged are nudged so that they abut. For each pair (`previous`, `indel`) of the same class with a gap between them (:193–202):

- **homopolymer case** (:203–215): if `previous` is a homopolymer, the reference between the two indels is the same homopolymer, and the read agrees, move `previous` right so it touches `indel`.
- **tandem-repeat case** (:216–234): walk `previous` right in steps of its own length as long as the next `previous.length` reference bases repeat the indel sequence; if that lands it exactly against `indel`, adopt the new position.

This is a **right** shift, applied only when it enables a merge that reduces the number of separate events — i.e. it is a parsimony move, not a normalisation move, and it is bounded by “does it let us merge”.

3.3.5 Step 4 — rebuild the CIGAR, merging on the way (:247–304)

Walk the (now shifted) indel list left to right, emitting CIGAR operations:

- leading **M** from the start of the read to the first indel;
- for each indel: if it sits **exactly** where the previous one ended and is the same class, extend the previous CIGAR op in place (:276–282) — this is the merge; otherwise emit **M** for the gap, then **I/D/N** (:283–291);
- trailing **M** to the end of the aligned region, then **softEnd** if any.

`indel.position < lastend` (an indel realigned **left of** a previous one) is treated as impossible and aborts the process (:271–275) — it would mean steps 1–3 violated their previous-indel guards.

3.3.6 Step 5 — did anything change?

The pass stringifies the CIGAR before and after and returns `cigar_after != cigar_before` (:337–341). `stablyLeftAlign` loops on this.

3.4 countMismatches (LeftAlign.cpp:345)

A straight CIGAR walk that counts positions where `read[rp] != ref[sp]` across **M/X/=** runs, advancing `sp` on **D/N**, `rp` on **I/S**. Used only as the `VERBOSE_DEBUG` invariant check described above. Note it takes the reference slice **by value** — a real (small) cost, but only in debug builds.

3.5 Worked example

Reference slice `A C **G T G T G T** A C`, read has one GT unit deleted, aligner placed the gap at the right edge:

```
pass 0 CIGAR: 2M 6M 2I? ...      ref:  A C G T G T G T A C
                                   read: A C G T G T - - A C      D at sp=6, len 2
step 1 (i=2):  sp=6 → left bases [4,6) = "GT" == indel.sequence "GT"  ✓ shift to sp=4
                sp=4 → left bases [2,4) = "GT" == "GT"                ✓ shift to sp=2
                sp=2 → left bases [0,2) = "AC" ≠ "GT"                  ✗ stop
pass 0 result: D now at sp=2 → CIGAR 2M 2D 4M 2M (changed ⇒ leftAlign returns true)
pass 1:        no further shift possible           ⇒ returns false
stablyLeftAlign: stable after 2 passes
```

Every read covering this deletion now reports it at reference offset 2.

4 Part II — Allele construction feeding realignment

4.1 updateAlignmentQueue (AlleleParser.cpp:1954): intake and the filter gauntlet

Per position, this pulls every alignment with `POSITION <= currentPosition` && `REFID == currentRefID` from the BAM stream (:1978, :2118). Each read runs a gauntlet **before** realignment or allele extraction (:2010–2044):

check	effect / parameter
read group → sample	read skipped if its <code>@RG</code> maps to no analysed sample (:2010)
duplicate	skipped unless <code>--use-duplicate-reads</code> (default: skipped) (:2016)
unmapped	skipped (:2022)
<code>AlignedBases == 0</code>	skipped (:2028)
secondary alignment	skipped — <code>SecondaryFlag()</code> (:2034)

out-of-order	warned + skipped if <code>ENDPOSITION < position (:2040)</code>
mapping quality	processed only if <code>MAPPINGQUALITY >= parameters.MQL0 (-m, default 1) (:2051)</code>

Then, still inside the gate:

1. `stablyLeftAlign` (if `leftAlignIndels`);
2. `capBaseQuality` if `--base-quality-cap != 0`;
3. `--skip-coverage` hard cap (:2075–2092): once per-position coverage exceeds the cap, the read is dropped **and** previously-registered reads/alleles overlapping that position are purged. Deterministic (no PRNG), unlike `--limit-coverage`.
4. `RegisteredAlignment ra(currentAlignment); registerAlignment(ra, ...)`;
5. **post-hoc read rejection** (:2104–2109): if the read produced no alleles, or `mismatches/SEQLEN > --read-max-mismatch-fraction`, or `mismatches > --read-mismatch-limit`, or `snpCount > --read-snp-limit`, or `indelCount > --read-indel-limit`, the whole `RegisteredAlignment` is popped (`rq.pop_front()`) — the read contributes nothing. All four limits default to effectively infinite.

4.2 `registerAlignment` (`AlleleParser.cpp:1329`): CIGAR → alleles

This is the read→allele converter. It walks the (possibly just-realigned) CIGAR and, for each operation, emits `Allele` objects via `makeAllele + ra.addAllele`.

Match/mismatch runs (M/X/=, :1422–1625). The CIGAR from most aligners uses `M` for both matches and mismatches, so `FreeBayes` re-derives them by comparing `read[rp]` against `currentSequence[csp]` base by base:

- a maximal run of equal bases becomes one `ALLELE_REFERENCE` allele, `quality = the read’s mapping quality (:1472, :1609)`;
- each mismatched base becomes a 1-bp `ALLELE_SNP` (or `ALLELE_NULL` if the read base is not ACGT) with that base’s `quality (:1516, :1533)`;
- a reference base of `N` always forces a mismatch (:1465);
- mismatches with `quality >= parameters.BQL2 (-Q, default 10)` increment `ra.mismatches` (used by the post-hoc read filter); `ra.snpCount` always increments (:1491–1494).

Adjacent SNPs are emitted individually here; MNPs and complex events are assembled later by `clumpAlleles`.

Deletion (D, :1626–1704). Emitted as `ALLELE_DELETION` of the reference substring, **unless** it is the first or last CIGAR op (aligner-reported terminal deletions are not trusted — :1681). Deletions carry no base qualities, so a proxy quality is synthesised from `l+2` bases of read quality centred on the event (:1638–1673): either the minimum quality (`--harmonic-indel-quality off`, i.e. default `useMinIndelQuality = true` uses `minQuality`) or a harmonic-sum-scaled sum. `ra.indelCount` increments.

Insertion (I, :1706–1773). Symmetric: `ALLELE_INSERTION` of the inserted read bases, proxy quality from `l+2` surrounding quality values, `indelCount++`.

Soft clip (S, :1776–1799). Emitted as `ALLELE_NULL` spanning the clipped reference footprint — it marks “this read says nothing here” so downstream code does not mistake a clip for reference support.

N (splice), H (hard clip), P (pad). `N` advances the reference pointer only (its `ALLELE_NULL` emission is commented out); `H/P` do nothing.

After the walk:

- if `--trim-complex-tail`, a trailing `M` run is split off a final complex allele (:1840–1856) — noted in-code as incomplete (the demoted allele’s `type` is not re-derived);

- `ra.start` / `ra.end` are set from the first/last allele's span (:1860);
- per-read mismatch/SNP/indel **rates** are computed and stored on every allele, each rate re-normalised to **exclude the allele itself** (:1898–1926) so that a downstream consumer can ask “how noisy is this read, ignoring the variant it supports”;
- `ra.clumpAlleles(...)` runs (:1938).

4.3 `makeAllele` (`AlleleParser.cpp:1171`): one allele + two side effects

Builds the CIGAR string for the event (NxM/X/I/D/N), grabs the matching reference substring, and constructs the `Allele`. Two non-obvious behaviours:

Haplotype-basis gating (:1222–1234) if `--haplotype-basis-alleles` is in use and this allele is not in the permitted set, it is **demoted to reference** (type $\rightarrow$ `ALLELE_REFERENCE`, quality $\rightarrow$ 0, sequence $\rightarrow$ the reference bases). The observation is not dropped; it is neutralised.

Repeat-boundary caching (:1236–1297, indels only) the allele records a `repeatRightBoundary` — how far right a tandem repeat or low-entropy stretch containing this indel extends. This is the piece most likely to be misread, so the source/sink of every term is stated explicitly:

- **The repeat catalog is reference-only and shared across every read.** `repeatCounts(pos - currentSequenceStart, currentSequence, 12)` (:1248) scans the whole in-memory reference contig for tandem units of length 1–12; the function takes no read argument at all (:4118–4144). The result is cached in `cachedRepeatCounts`, keyed *only* by `pos` (`AlleleParser.h:297`), so the first allele that lands on a given `pos` populates it and every later `makeAllele` call — from any other read — at that same `pos` reuses it verbatim (:1246–1250); it is never recomputed per read or per allele.
- **Whether a cached unit applies to this allele can depend on the read.** The gate is `isRepeatUnit(alleleseq, repeatunit)` (:1257, :4170–4182, “does `alleleseq` tile exactly with `repeatunit`”), where `alleleseq` is the allele's own sequence: for `ALLELE_DELETION` it is `refSequence` (:1244, itself a reference substring, :1204) — so deletions are reference-only end to end. For `ALLELE_INSERTION` it is `readSequence` (:1242) — the read's actual inserted bases — so an insertion only inherits a cached repeat unit if its own inserted sequence is built from copies of that unit.
- **Entropy extension is reference-only:** the while at :1282–1290 extends `repeatRightBoundary` while `entropy(currentSequence.substr(...)) < --min-repeat-entropy` (default 1.0), bounded by the read's alignment end (never past it — no point extending where no read can be a covering haplotype observation).
- **One more read-dependent bump, independent of the catalog:** :1292–1296 — if the reference bases immediately right of `pos` equal the read's inserted bases, extend to `pos + length + 1`. This is “repeat in the read but not in the reference” and does not go through `cachedRepeatCounts` at all.

Crucially, none of this reads `ra.alleles`, `registeredAlignments`, `registeredAlleles`, or any count of neighbouring events — every `repeatRightBoundary` is a function of (`pos`, `type`, `alleleseq`) and the reference alone. Two indel alleles at the same `pos` with the same `alleleseq` get the identical boundary; the presence of *other* nearby indels — however many — has no effect on this computation. See §5.4 for where multiple nearby events actually get combined.

4.4 `addAllele` / `clumpAlleles` / `mergeAllele`: complex-event assembly

`addAllele` (:1000) just appends and ORs the type bit. The work is in `clumpAlleles` (:1013), run once per read after the CIGAR walk, gated on `--max-complex-gap` (default 3):

1. mark runs of alleles to merge (:1018–1041): any window `last-curr-next` where `last` and `next` are non-reference and the middle is either non-reference or a reference gap $\leq$ `maxComplexGap`. This turns `SNP-2M-INS-1M-SNP` into one `ALLELE_COMPLEX`.
2. merge marked runs with `Allele::mergeAllele (:1047–1049)`.
3. re-attach one flanking base to indels that ended up at a clump edge (:1063–1090, via `subtractFromEnd/addToStart/subtractFromStart/ addToEnd`) so every indel has a matched base on each side (VCF-friendly).
4. drop 0-length alleles (`isEmptyAllele, length == 0`).

`Allele::mergeAllele (Allele.cpp:1480)` concatenates sequences and cigars, averages qualities — and does `length += newAllele.length; // hmmm`. That comment is the FreeBayes author’s own flag: for a merged `ALLELE_COMPLEX` the `length` member becomes the **sum of component lengths**, which is not `alternateSequence.size()` and not what `updateTypeAndLengthFromCigar` would assign.

This is the exact field responsible for a real, observed `LEN=9-vs-8` inconsistency between two representations of what should be the same complex allele — but the bug’s effect is not permanent for every occurrence. `updateTypeAndLengthFromCigar()` has exactly two call sites in the whole codebase: inside `Allele::subtract()` (`Allele.cpp:1225`, backing `subtractFromStart/subtractFromEnd`) and inside `Allele::add()` (:1324, backing `addToStart/addToEnd`) — both used by the flanking-base step above *and* by `fitHaplotype` (§5.7). Any allele that passes through either afterwards has its `length` correctly re-derived from (`type, alternateSequence, cigar`); `Allele::update()` (:28–42, called by `fitHaplotype`) does *not* touch `length` at all. So the bug only survives to the final candidate list on an allele that is merged by `mergeAllele` and then never subsequently trimmed or re-merged — see §5.7 for exactly when that happens and why it explains an 8-vs-9 split without needing to invoke anything binary-layout-dependent.

4.5 `getAlleles (AlleleParser.cpp:3685)`: bucketing

Given the flat `registeredAlleles` pointer vector, `getAlleles` selects those overlapping `currentPosition` (with the length/haplotype rules at :3714–3734) and buckets survivors into `samples[sampleID][currentBase] → vector<Allele*>`. Filters applied here: allele quality $\geq$ `--min-base-quality` (BQLO, default 0), `currentBase != "N"`, non-empty alternate sequence, and the allele-type mask (`--no-snps / --no-indels / --no-mnps / --no-complex`). An allele is marked `processed` so it is not re-emitted at a later position.

5 Part III — `buildHaplotypeAlleles (AlleleParser.cpp:3214)`

5.1 Why it exists

Consider a 1-bp deletion inside a 10-bp homopolymer. At the deletion’s left-most (canonical) position, “reference” and “1-bp deletion” differ by a single base — easy to confuse under sequencing error. If instead we genotype the **entire homopolymer** as one block, the reference haplotype is `AAAAAAAAAA` and the variant haplotype is `AAAAAAAAA`; a read either matches one or the other over all 10 bases, and the discrimination is unambiguous. `buildHaplotypeAlleles` decides how wide that block must be and rewrites every overlapping read as a single allele across it.

5.2 Seeding the window length (:3223–3239)

`haplotypeLength` starts at 1 and is bumped up by, for each non-reference candidate allele:

- `allele.referenceLength` (a complex/MNP event already spans >1 ref base);
- `allele.repeatRightBoundary - currentPosition` (the cached repeat extent from `makeAllele`, §4.3).

If there are no registered alignments, it returns immediately (:3242).

5.3 The growth loop (:3245–3310)

A `do ... while` (`haplotypeLength != oldHaplotypeLength`) `fixpoint`:

1. clear `samples`; walk `registeredAlignments[i]` for `i` in (`currentPosition`, `maxAlignmentEnd`) and, for every read that **starts or ends** inside the current window, call `ra.fitHaplotype` and collect its alleles (:3266–3280);
2. `getAlleles + groupAlleles + genotypeAlleles` at the current `haplotypeLength` (:3282–3285);
3. for each resulting non-reference allele, if `allele.end` or `allele.repeatRightBoundary` reaches past `currentPosition + haplotypeLength`, grow `haplotypeLength` to cover it (:3286–3309);
4. repeat until the length stops changing.

This loop only has visibility into reads that have already been streamed in (`POSITION <= currentPosition`, §4.1): it sees reads that **end** inside the growing window, because those were necessarily registered at or before `currentPosition`, but it cannot see reads that only **start** somewhere inside the window and have not been streamed in yet. A read whose sole contribution to this haplotype block is a variant that starts a few bases to the right of `currentPosition` is invisible to this loop’s window-growth decision on the first pass — it is only picked up once the position sweep itself reaches that far. This is a structural consequence of FreeBayes’s single-pass, streaming design (§7), not a bug as such, but it means the haplotype window’s final size can depend on the order positions are swept in.

5.4 Worked example: several nearby indels

A question worth answering precisely, because it is easy to guess wrong: *deletion at position P, plus ten separate insertion events nearby — how do their `repeatRightBoundary/referenceLength` values combine when the window grows?*

Per allele, nothing combines yet. Each of the eleven `makeAllele` calls (§4.3) is independent — the function reads only (`pos`, `type`, `alleleseq`) and the reference-derived, position-keyed `cachedRepeatCounts` cache; it never looks at how many other indel alleles exist nearby. Two alleles with the same (`pos`, `alleleseq`) get the identical `repeatRightBoundary`; alleles with different sequences or positions get independently-computed ones — in neither case does the **count** of nearby events enter the computation.

Combination happens here, in the seed and growth steps above, and it is a running max, not a sum or a count. The seed loop (§5.2) folds `allele.referenceLength` and `allele.repeatRightBoundary - currentPosition` across every candidate allele with `haplotypeLength = max(haplotypeLength, ...)`; the growth loop (§5.3, :3286–3309) does the same with `hapend = max(allele.position + allele.referenceLength, allele.repeatRightBoundary)` per candidate, taking the largest `hapend` seen. So with the deletion and ten insertions all present as distinct candidates, `haplotypeLength` ends up equal to whichever *single* allele reaches furthest right — not their sum, and not a function of how many there are.

One subtlety the “eleven alleles” framing glosses over: the `alleles` list these loops iterate is the *deduplicated candidate* list from `genotypeAlleles()` — one entry per distinct alt sequence, taking `alleles.front()` of each base-group as the representative (§5.7). If several of the ten insertions are really repeat *observations* of the same event (ten reads reporting the same inserted sequence at the same position), they collapse to **one** candidate before either loop runs, and only that one entry’s `repeatRightBoundary` participates in the `max`. Only genuinely distinct alt sequences/positions show up as separate entries — in the fully-distinct case, all eleven (ten insertions + the deletion) contribute to the running maximum.

5.5 `fitHaplotype` (`AlleleParser.cpp:3040`): squashing a read into one allele

Given a haplotype window [`haplotypeStart`, `haplotypeEnd`) and a read’s allele list, `fitHaplotype`:

1. bails (return false, restoring savedAlleles) unless the read has an allele overlapping the window start and one overlapping the window end (:3063–3086);
2. refuses to assemble across an ALLELE_NULL (:3116) — a clip/ambiguous base inside the window makes the read a non-observer;
3. trims the boundary alleles so they start/end exactly on the window (subtractFromStart/ subtractFromEnd, :3119–3145);
4. merges everything between into one allele via addToStart, taking the **minimum** quality across the merged pieces (:3148–3160), and squashes the consumed alleles (length = 0, later erased).

The result is one allele whose alternateSequence is the read’s actual bases over the whole window — exactly what the genotyper compares against the reference haplotype.

5.6 Complete vs partial observations

Complete observations reads that fully span the window (ra.start <= start && ra.end >= end), collected by getCompleteObservationsOfHaplotype (:3576). These are the first-class evidence.

Partial observations reads that overlap the window but do not span it, collected by getPartialObservationsOfHaplotype (:3620) — only when --use-partial-observations is set and haplotypeLength > 1. A partial read consistent with several haplotype alleles contributes fractionally to each (Sample::assignPartialSupport; partialObservationSupport : map<Allele*, set<Allele*>>). buildHaplotypeAlleles then demotes a “complete” observation to partial if it turns out to support more than one allele (:3477–3491).

5.7 Re-derivation, what sort/unique actually deduplicates, and the LEN bug precisely

This section was tightened after two rounds of source-verified follow-up questions on the LEN=9-vs-8 divergence at the real MY2693 record (contig position 79745, REF=TCAACAG ALT=ACATGAAA CIGAR=1X2M2I1M1D1M1X). Everything below is traced against the source directly; nothing here was pulled from the actual BAM records for that site (not available in this environment) — where that would matter, it is flagged explicitly.

5.7.1 What sort + unique on registeredAlleles deduplicates

Inside the do/while growth loop (:3259–3310), for every read overlapping the current window, ra.fitHaplotype(...) is called and then *every* element currently in ra.alleles is pushed onto registeredAlleles (:3274–3277) — regardless of fitHaplotype’s return value, and regardless of whether ra.alleles changed at all. Two things matter:

- registeredAlleles is never cleared inside this loop (only samples.clear() at :3263) — it is only appended to, iteration after iteration, until it is rebuilt from scratch after the loop settles (:3537–3548, well outside the do/while).
- as haplotypeLength grows across iterations, the **same** ra typically still overlaps the (now wider) window and gets processed again. If fitHaplotype finds nothing to do for it this time — the common case is its early exit at :3074/:3084 when no allele in ra.alleles yet reaches the new window edge, which returns false **without touching alleles at all** — the exact same, unmodified Allele* addresses get pushed onto registeredAlleles a second (or third, ...) time.

This is the literal pointer duplication that sort(registeredAlleles...); registeredAlleles.erase(unique(...), ...) (:3413, :3509) removes: the same object, pushed more than once because the outer scan revisits it every time the window grows without something new happening to it. Sorting by raw address (std::less<Allele*>, no comparator given) is only there to make std::unique’s adjacent-equal check work — unique compares Allele* values with ==, i.e. pointer identity, so it can only remove an address that is literally repeated. **It cannot and does not**

collapse two different `Allele` objects that merely have matching content (same `currentBase`, different `length`) — that is a structurally different problem, addressed next.

5.7.2 Where “same observation” is actually decided — and what it doesn’t check

The content-based grouping the previous paragraph says `sort/unique` is *not* doing happens one step later, in `getAlleles` (`AlleleParser.cpp:3742`):

```
samples[allele.sampleID][allele.currentBase].push_back(*a);
```

This buckets survivors of `registeredAlleles` by matching the `currentBase` **string** — this is the actual “these are the same observation” decision, and it is a distinct mechanism from the pointer-identity dedup above; one groups by content (a string), the other by address (a pointer). Nothing at this bucketing step — nor in `groupAlleles` (`Sample.cpp:274`, which just concatenates each sample’s per-base bins into `alleleGroups[base]`) — checks that every `Allele*` landing in the same bucket agrees on `length`, `cigar`, or `referenceLength`. `AlleleParser::genotypeAlleles` (:3878) then reads `alleles.front()` of that bucket as the representative without verifying that assumption. This is the actual gap: matching `currentBase` is treated as sufficient to make all bucket members interchangeable, and nothing enforces it.

5.7.3 Why two different `length` values can legitimately reach the same bucket

Given the above, the remaining question is why the bucket for `ACATGAAA` would ever contain objects with genuinely different `length` — not merely duplicate pointers to the same one. Tracing every place `length` is assigned in `Allele.cpp` gives an exhaustive, two-branch answer:

- `Allele::mergeAllele` (:1484, §4.4) sets it via the unaudited `length += newAllele.length`, when `RegisteredAlignment::clumpAlleles` first assembles a complex event per-read, before `buildHaplotypeAlleles` runs at all.
- `updateTypeAndLengthFromCigar()` — which correctly sets `length = alternateSequence.size()` for `ALLELE_COMPLEX` (:1403) — has exactly two call sites in the entire codebase: inside `Allele::subtract()` (:1225) and inside `Allele::add()` (:1324). Both are reached only via `subtractFromStart/subtractFromEnd/addToStart/addToEnd`, which are called from `clumpAlleles`’s flanking-base step (:1075–1090) and from `fitHaplotype` (:3126–3157) — and nowhere else. `Allele::update()` (:28–42, called from `fitHaplotype:3180`) touches `currentBase`, `quality` and `basesLeft/basesRight` — never `length`.

So a complex allele’s `length` is correct **if and only if** it has passed through `subtract()/add()` at some point after `mergeAllele` built it. Inside `fitHaplotype`, that happens only when the read’s own clumped allele does **not** already exactly span [`haplotypeStart`, `haplotypeEnd`): trimming one or both ends (:3120–3145) or merging with a neighbour (:3148–3160, the `while (a != b)` loop) both go through `subtract/add` and self-heal `length`. A read whose own `clumpAlleles`-assembled allele already starts exactly at `haplotypeStart` (:3120, “nothing to do!”) and ends exactly at `haplotypeEnd` (:3131, “nothing to do!!!!”), with nothing between `a` and `b` to merge (`a == b`, so the merge loop runs zero times), passes through `fitHaplotype` completely untouched — its `length` is whatever `mergeAllele` gave it, verbatim, uncorrected.

This is a plain, deterministic explanation for an 8-vs-9 split with **no** need to invoke undefined behaviour: whichever read’s own natural event boundary happens to already coincide with the final converged haplotype window keeps `mergeAllele`’s (possibly stale) sum; every other read supporting the same allele, needing any trim or re-merge to fit that window, gets `length` correctly re-derived. **Which** value ends up as `alleles.front()` — and therefore in `LEN` — is then exactly the address-order question the sort at :3413/:3509 leaves unresolved (§7). Confirming that this specific mechanism (rather than something else) is what happened for the 31 reads supporting the 79745 record would

need runtime instrumentation on that BAM, which is not available here — flagged rather than asserted as confirmed.

5.7.4 A separate, weaker hazard in the same function

`fitHaplotype`’s own comment at :3164–3165 — “*this operation requires independent removal of references to these alleles (e.g. `registeredAlleles.clear()`)*” — is the author flagging that `alleles.erase` and the subsequent `push_back/sort` in this function (:3164–3171) can invalidate or move `Allele*` values that something `else` is still holding. `registeredAlleles` is exactly such a holder. This is a real, author-acknowledged pointer-invalidation hazard in the same code path, but it is not needed to explain the 79745 divergence (the previous subsection is sufficient and fully deterministic) and has not been separately confirmed to be operative here — noted for completeness, not asserted as the cause.

6 Coordinate systems

Three coordinate frames coexist; mixing them up is the most common source of off-by-one bugs in this area.

frame	0/1-based	used by
<code>currentPosition</code> , <code>Allele.position</code> , <code>BedTarget.left/right</code>	0-based, end-inclusive	the whole engine internally
BAM POSITION / SeqLib GenomicRegion	0-based, half-open	read intake, <code>SetRegion</code>
<code>-r chr:S-E</code> on the CLI	1-based input	<code>loadTargets</code> parsing (:722)
VCF POS	1-based	<code>var.position = currentPosition + 1</code> (<code>ResultData.cpp:61</code>)
tabix region for <code>-@</code> input VCF	1-based, fully closed	<code>loadTarget</code> (:2742)

`currentSequence` holds the **entire** contig in memory; `currentSequenceStart` is always 0, so `Allele.position` is an absolute offset into it. The realignment reference slice is `currentSequence.substr(readStart, readLen)`, so inside `leftAlign` positions are relative to the **read**, not the contig.

7 Known fragilities and non-determinism

This section is the running list of places where the pipeline’s output can depend on something other than the input data.

1. **Global PRNG for `--limit-coverage`** (`freebayes.cpp:109, :189`). The only `srand/rand` in the codebase. Seeded once, at process start; the downsampling decision at a given site depends on how many `rand()` calls preceded it in the run. Consequently, splitting a run into several processes over sub-regions of the same input (each with its own seed and its own, shorter sequence of downsampling decisions) can select different reads to drop than a single process covering the whole input would, at any site where the coverage cap fires — so the two can disagree at those sites even on identical data.

2. **std::sort of vector<Allele*> by address** (AlleleParser.cpp:3413, :3509). Sorting pointers with no comparator sorts by raw memory address. Everything downstream that iterates the result and picks `.front()` as a group’s representative — `genotypeAlleles (:3878)` among them — is therefore sensitive to heap layout: which of several same-content observations is treated as canonical can depend on where their objects happen to sit in memory, which can differ between otherwise-identical runs (different binary, different build, different allocator state). `cigar`, `referenceLength`, `length` and `position` are all taken from `.front()` this way and could in principle diverge for any base-group whose constituent objects disagree — see §5.7 for a confirmed case (the `LEN` field) and its precise mechanism.
3. **Allele::mergeAllele’s length += newAllele.length; // hmmm** (Allele.cpp:1484) — the comment is the original author’s own flag. A merged complex allele’s `length` is left inconsistent with its sequence, and nothing re-derives it before it may be read back out (§5.7 traces exactly when this does and doesn’t get self-corrected downstream).
4. **leftAlign non-convergence.** The pipeline call (AlleleParser.cpp:2057) does not check `stablyLeftAlign`’s return value, so a read that fails to stabilise within 20 iterations is silently used with its last intermediate CIGAR. `bamleftalign` (the standalone tool) does warn.
5. **VERBOSE_DEBUG-only invariant.** The “mismatch count unchanged by realignment” assertion (LeftAlign.cpp:403) and every `DEBUG2` call are compiled out unless the codebase is built with `VERBOSE_DEBUG` defined, which ordinary release builds do not do. Realignment correctness is not checked at runtime in a normal build.
6. **--trim-complex-tail type staleness** (AlleleParser.cpp:1846, in-code `FIXME`): an allele split by tail-trimming keeps `type == ALLELE_COMPLEX` even when it is no longer complex. Off by default.

8 Appendix — parameter defaults touching this pipeline

From `Parameters.cpp`; an example long-read calling profile overrides only the four marked ★.

flag	default	role
<code>-O / --dont-left-align-indels</code>	left-align ON	disables Part I
<code>-m / --min-mapping-quality</code>	1	read intake
<code>-q / --min-base-quality</code>	0	<code>getAlleles</code> allele drop
<code>-Q / --mismatch-base-quality-threshold</code>	10	counts toward <code>ra.mismatches</code>
<code>--read-max-mismatch-fraction</code>	1.0 (off)	post-hoc read reject
<code>-U / --read-mismatch-limit</code>	10 ⁷ (off)	post-hoc read reject
<code>--read-snp-limit / --read-indel-limit</code>	10 ⁷ (off)	post-hoc read reject
<code>-E / --use-duplicate-reads</code>	off (dups removed)	read intake
<code>--max-complex-gap / --haplotype-length</code>	<code>maxComplexGap = 3</code>	both option 'E'; the latter falls through to set <code>maxComplexGap</code> (<code>Parameters.cpp:784</code>). <code>clumpAlleles</code> merge width / haplotype seeding
<code>--min-repeat-size</code>	5	<code>makeAllele</code> repeat-boundary
<code>--min-repeat-entropy</code>	1.0	<code>makeAllele</code> repeat-boundary
<code>-C / --min-alternate-count ★</code>	2	candidate-allele filter
<code>-F / --min-alternate-fraction ★</code>	0.05	candidate-allele filter
<code>--limit-coverage ★</code>	0 (off)	PRNG downsample

<code>--pooled-continuous ★</code>	off	alt-selection + likelihood branch
<code>-g / --skip-coverage</code>	0 (off)	deterministic coverage cap
<code>-0 / --standard-filters</code>	—	sets <code>-m 30 -q 20</code>
<code>-j / --harmonic-indel-quality</code>	off (uses min)	indel proxy quality

9 Implications for panFreebayes

Everything above describes FreeBayes itself. This closing section is project-specific: it records where a downstream extraction effort (panFreebayes, which lifts this calling engine out of the FreeBayes CLI for use outside whole-genome/whole-BAM invocation) made a deliberate choice about one of the properties documented above, rather than that being a property of FreeBayes in general.

- **PRNG reseeding (§7, global PRNG item).** panFreebayes re-seeds the PRNG on every call and never subdivides an input across processes, so its output is deterministic and matches what a single FreeBayes process covering the same input would produce — not what a multi-process/chunked invocation would.
- **Streaming window-growth blindness (§5.3).** panFreebayes preserves this behaviour exactly rather than working around it — it does not pre-register reads ahead of the position sweep, so its haplotype-window growth has the same blind spot described there.
- **The LEN non-determinism (§5.7).** This has been fixed in the engine panFreebayes builds on, by deriving `length` from `(type, altseq, cigar)` instead of trusting whichever object `.front()` happens to select. The underlying address-order `sort` (§7) has not been removed, so the same class of issue remains possible for the other fields — `cigar`, `referenceLength`, `position` — that are still read from `.front()`.